

# Running Python Code (CodeGrade)

 [\\_ \(https://github.com/learn-co-curriculum/python-p3-running-python-code\)](https://github.com/learn-co-curriculum/python-p3-running-python-code)  [\\_ \(https://github.com/learn-co-curriculum/python-p3-running-python-code/issues/new\)](https://github.com/learn-co-curriculum/python-p3-running-python-code/issues/new)

## Learning Goals

- Run Python code from a file.
- Log output to the terminal.
- Run Python code from the Python shell.
- Create a `pipenv` virtual environment.
- Run `pytest` tests.

---

## Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
  - **Python Shell:** an interactive interpreter that can be accessed from the command line.
  - **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
  - **Exception:** a type of error that can be predicted and handled without causing a program to crash.
  - **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
  - **Function:** a named code block that performs a sequence of actions when it is called.
  - **Scope:** the area in your program where a specific variable can be called.
-

# Introduction

In this lesson, you'll get some practice running Python code, and see a few different ways to check what your code is doing. Make sure to code along to get comfortable in this new environment!

---

## Creating a Python Application

Let's dive right in. To get started on any new Python application, the first thing we need is a file. Fork and clone this lesson from GitHub, then create a new file in this repo's `lib/` directory called `app.py`. In this file, add the following:

```
# lib/app.py
print("Hello world!")
```

`print()` is a built-in Python function that will output a string of text to the terminal. It's the Python equivalent of `console.log()` in JavaScript. It will print the string "Hello world!" along with a line break at the end.

The line above `print()` is a Python comment. In Python, any line that starts with a `#` won't be executed by the interpreter. This is the Python equivalent of `//` in JavaScript.

---

## Running Python Applications

Unlike JavaScript, you won't be running Python applications in the browser. Instead, you'll need to use the Python interpreter to run your code from the terminal. You can check which version of Python you're using with this command:

```
$ python --version
Python 3.8.13
```

To run the application, enter the command `python filename.py`, where `filename.py` is the relative path to the file you'd like to run:

```
$ python lib/app.py  
Hello world!
```

Congrats on running your first Python application! 🎉

---

## Displaying Data with "print()"

Now that we have a place to write some code, let's explore a few of Python's built-in options for displaying data in different ways.

### Standard print() Statements

By default, `print()` includes a newline character at the end of your string. Try entering several `print()` statements in a row in `lib/app.py`:

```
# lib/app.py  
print("Hello world!")  
print("Hello sun!")  
print("Hello sky!")
```

What do you see when you execute `lib/app.py` from the command line?

```
$ python lib/app.py  
# => Hello world!  
# => Hello sun!  
# => Hello sky!
```

---

### Choosing your own print() Ending

Let's say you're writing a full paragraph and don't need a newline character after every sentence. `print()` can accommodate that through its optional `end` parameter. Try modifying your `print()` statements as follows:

```
# lib/app.py
print("Hello world!", end=" ")
print("Hello sun!", end="!! ")
print("Hello sky!", end="!!!\n")
```

What do you see when you execute `lib/app.py` from the command line with these new `end` strings?

```
$ python lib/app.py
# => Hello world! Hello sun!!! Hello sky!!!!
```

`end` can be a string of any length, including characters like the newline `\n`.

---

## Exploring Python with the Python Interpreter

Python comes with a command line interpreter (often called the "Python shell") for running a Python REPL (read-evaluate-print-loop) in the terminal, which provides similar functionality to the browser console that you're familiar with from JavaScript. Using the Python shell is a great way to quickly test out some code, or check your syntax, without needing to run an entire application.

To use the Python shell, go into the terminal and enter `python` :

```
$ python
Python 3.8.13 (default, Jun  2 2022, 15:59:12)
[Clang 13.1.6 (clang-1316.0.21.2.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

This gives you a prompt where you can enter Python code. Try entering in `print("Hello Python shell")` :

```
$ python
Python 3.8.13 (default, Jun  2 2022, 15:59:12)
[Clang 13.1.6 (clang-1316.0.21.2.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello Python shell")
# => Hello Python shell
```

Try running a few more expressions in the Python Shell:

```
>>> first_number = 7
>>> print(first_number)
# => 7
```

In the code above, we've declared a **local variable** called `first_number` and assigned it a value of `7`. When we tell the Python shell to `print(first_number)`, we see our local variable's value on the next line.

In Python, it's convention to use underscores ( `_` ) to separate words in variables. This is referred to as **snake case** (as opposed to **camel case**, which is the convention in JavaScript).

You can exit the Python shell by typing `exit()`, or pressing `ctrl + d`.

---

## Running `pytest` Tests

All the lessons in the Python curriculum use the `pytest` library for testing your Python code.

In this lesson, you'll see a `testing/` folder with one file, `app_test.py`. `app_test.py` is where we've defined tests specifically for this lesson.

`pytest` is a Python library (the Python equivalent of an npm package) that provides a very simple and clean way to write tests.

---

## Installing `pytest`

To install `pytest` and any other required libraries, simply navigate to a folder with a `Pipfile` and enter `pipenv install`:

Now that your `pipenv` virtual environment is ready to use, enter `pipenv shell` to start working:

Let's take one of these tests as an example to see `pytest` 's syntax:

6/10

```
assert(path.exists("lib/app.py"))
```

`pytest` expects a **function** with `test` in its name within the `testing` folder. We will discuss functions later on in this module. For now, all that you need to know is that a function performs an action when it's called.

Normally, you would have to call the function yourself with `test_app_py_exists()`, but in this case `pytest` is calling all `test` functions for you.

Delete the file `lib/app.py` to start from scratch so we can try getting these tests to pass.

To run the tests, you will simply run `pytest` from inside of your `python-p3-running-python-code` directory.

```
$ pytest
===== test session starts =====
platform darwin -- Python 3.10.4, pytest-7.1.2, pluggy-1.0.0
rootdir: /Users/benbotsford/Documents/python-p3-running-python-code
collected 3 items

lib/app.py exists in lib directory F           [ 33%]
lib/app.py is executable F                     [ 66%]
lib/app.py prints "Hello World! Pass this test, please." F [100%]

===== FAILURES =====

...

===== short test summary info =====
FAILED lib/app.py exists in lib directory - AssertionError: assert False
FAILED lib/app.py is executable - FileNotFoundError: [Errno 2] No such file or di...
FAILED lib/app.py prints "Hello World! Pass this test, please." - FileNotFoundErr...
===== 3 failed in 0.02s =====
```

`pytest` provides all kinds of useful information about what went right and wrong with our code, so make sure to spend your time reading all the output here! It will tell you:

- Which tests passed/didn't pass
- Why each failing test failed (the difference between the expected output and the actual output)
- The line number of the failing test

You can also use the `-x` flag to tell `pytest` to stop running after the first failing test. This technique is helpful for focusing your attention on one problem at a time:

```
$ pytest -x
```

```
===== test session starts =====
platform darwin -- Python 3.10.4, pytest-7.1.2, pluggy-1.0.0
rootdir: /Users/benbotsford/Documents/python-p3-running-python-code
collected 3 items
```

```
lib/app.py exists in lib directory F
```

```
===== FAILURES =====
----- TestAppPy.test_app_py_exists -----
```

```
self = <app_test.TestAppPy object at 0x103743a90>
```

```
def test_app_py_exists(self):
    """
    exists in lib directory
    """
```

```
> assert(path.exists("lib/app.py"))
```

```
E AssertionError: assert False
```

```
E + where False = <function exists at 0x10292e0e0>('lib/app.py')
```

```
E + where <function exists at 0x10292e0e0> = path.exists
```



```
testing/app_test.py:16: AssertionError
===== short test summary info =====
FAILED lib/app.py exists in lib directory - AssertionError: assert False
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 failed in 0.01s =====
```

---

## Instructions

To finish this lab, use the `print()` in the `lib/app.py` file as described by the tests:

- Use `print()` to display the text on one line: "Hello World! Pass this test, please."

Using `pytest` will run the tests. After they are passing, sync your progress using Git. When your tests are all passing and your work is synced, the lab is complete!

---

## Conclusion

This lesson covered a good amount of material, but now you should be familiar with running code in a Python `pipenv` environment, and using tools like the Python Shell and `pytest` as well as built-in methods like `print()` to understand what happens when your Python code is running. You'll need all these tools going forward, so make sure to get practice with all of them as you progress through this phase!

---

## Resources

- [Python print\(\) function](https://www.w3schools.com/python/ref_func_print.asp) ➞ [https://www.w3schools.com/python/ref\\_func\\_print.asp](https://www.w3schools.com/python/ref_func_print.asp)
- [Python Getting Started](https://www.w3schools.com/python/python_getstarted.asp) ➞ [https://www.w3schools.com/python/python\\_getstarted.asp](https://www.w3schools.com/python/python_getstarted.asp)
- [pipenv Documentation](https://pipenv.pypa.io/en/latest/) ➞ <https://pipenv.pypa.io/en/latest/>
- [pytest Documentation](https://docs.pytest.org/en/7.1.x/) ➞ <https://docs.pytest.org/en/7.1.x/>

This tool needs to be loaded in a new browser window

Load Running Python Code (CodeGrade) in a new window